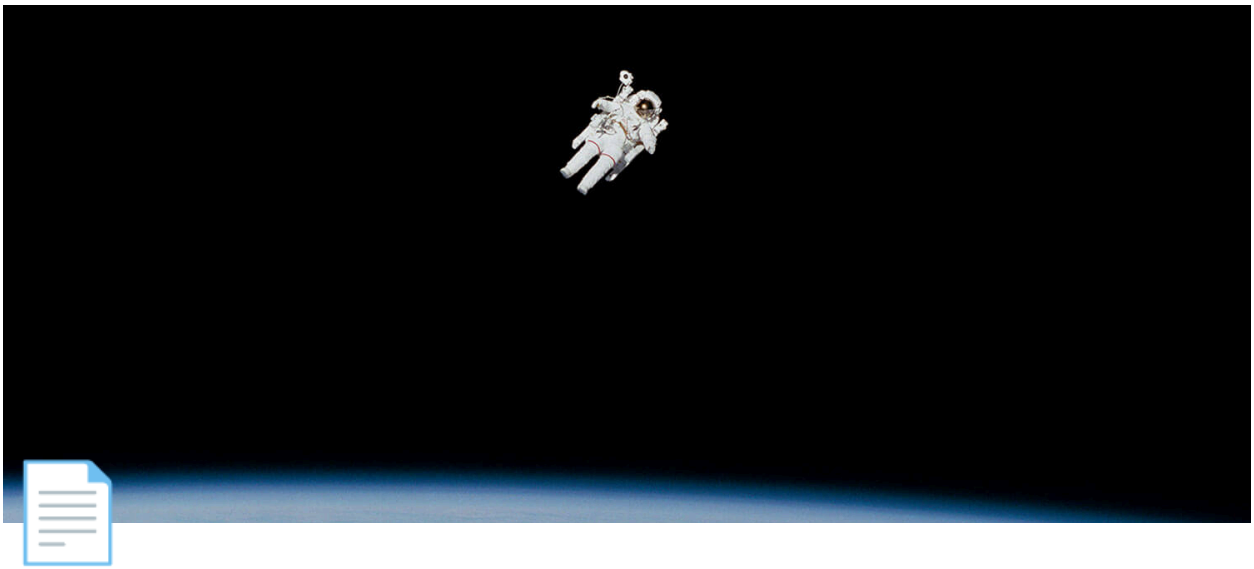

THE LIFE PROJECT — FINAL BACKEND FOUNDATION DOCUMENT (Refined Final Version for Google AI Studio)

THE LIFE PROJECT — FINAL BACKEND FOUNDATION DOCUMENT

Refined Final Version for Google AI Studio

---

1. DOCUMENT PURPOSE

This document defines the final backend foundation for The Life Project.

It is aligned with:

- selected language: Python
- selected backend framework: FastAPI
- selected database: PostgreSQL

- selected frontend: Next.js / React
- selected authentication: Google Login
- selected AI scope: onboarding + reflection + task generation
- selected phase scope: Phase 1 MVP only
- selected deployment: Vercel + managed backend host

This document is the final architectural direction for the new build.

It replaces scattered earlier assumptions and removes unnecessary complexity.

---

2. PRODUCT DEFINITION

The Life Project is a structured life operating system.

Its purpose is to help users:

- feel less empty
- feel more calm
- regain direction
- reduce digital distraction
- improve emotional balance
- build discipline
- discover meaning and purpose
- develop peace and inner stability

The backend must support this transformation safely and clearly.

Important:

The product is not a medical diagnosis system.

It is a behavioral, reflective, and guidance-based system.

---

3. CORE USER PROBLEMS TO SUPPORT

The backend must support users dealing with:

- emptiness
- lack of meaning
- lack of direction
- anxiety
- overthinking
- emotional imbalance
- digital distraction
- weak discipline
- poor sleep
- low inner peace
- loneliness / feeling disconnected

These problems must be handled through structured questions, reflection, behavior tracking, task generation, and calm support.

---

4. CORE PRODUCT PRINCIPLE

The system should not only store data.

It should do this:

- Input → understand the user
- Interpretation → identify their current state
- Action → generate helpful next steps
- Support → encourage calm, reflection, and consistency
- Progress → show growth over time

Core principle:

The app should feel human, supportive, and structured.

Not robotic.

Not clinical.

Not preachy.

---

5. HUMAN-LIKE LANGUAGE PRINCIPLE (MANDATORY)

All user-facing language inside the app must feel:

- friendly
- calm
- simple
- respectful
- emotionally safe
- human-like
- supportive
- clear

The app must not sound:

- cold
- robotic
- overly technical
- judgmental
- harsh
- overly clinical

This applies to:

- onboarding questions
- analysis summaries
- task suggestions
- reflection prompts
- meditation recommendations
- progress feedback

6. USER LANGUAGE RULES

Use language like:

- “How have you been feeling lately?”
- “Do you find it hard to stay focused these days?”
- “Does your day often feel directionless?”
- “Would a short calm session help you right now?”
- “You’ve shown effort today. Let’s build on that.”

Avoid language like:

- “Your productivity score is critically low.”
- “You are emotionally unstable.”
- “You have severe dysfunction.”
- “You are failing at discipline.”

Rule:

The app should feel like a wise, calm guide.

Not like a machine.

---

7. FINAL SELECTED STACK

Frontend:

- Next.js
- React
- Tailwind CSS
- TanStack Query
- Zustand (optional)

Backend:

- FastAPI

- Pydantic
- SQLAlchemy
- Alembic

Database:

- PostgreSQL

Authentication:

- Google Login

AI:

- Gemini API

Deployment:

- Frontend on Vercel
- Backend on Render or Railway
- PostgreSQL on Neon / managed PostgreSQL

---

8. WHY THIS STACK IS FINAL

This stack is correct because:

- Python matches your AI/ML learning direction
- FastAPI is clean and structured
- PostgreSQL is best for learning real database design
- SQLAlchemy gives strong model structure
- Alembic gives migration control
- Gemini API allows AI integration without training ML models
- Next.js remains a strong frontend base
- deployment path is practical

Direct override:

Do not use a more complicated stack now.

Do not add infrastructure that you do not yet need.

---

9. FINAL BACKEND ARCHITECTURE STYLE

Architecture style:

- modular
- layered
- API-first
- backend as source of truth

Flow:

Route → Service → Engine → Database / Model → Response

Responsibilities:

- Route: receives request, returns response
- Service: coordinates logic
- Engine: decision-making and pure logic
- Model: persistent database structure
- Schema: request/response validation
- Integration: external APIs like Gemini and Google auth

---

10. FINAL BACKEND FOLDER STRUCTURE

```
the_life_project_backend/
│
├── app/
│   ├── main.py
│   │
│   ├── core/
│   │   ├── config.py
│   │   ├── database.py
│   │   ├── security.py
```

```
│   │   └── dependencies.py
│   │
│   ├── api/
│   │   └── v1/
│   │       ├── auth.py
│   │       ├── onboarding.py
│   │       ├── tasks.py
│   │       ├── reflection.py
│   │       ├── progress.py
│   │       ├── meditation.py
│   │       └── users.py
│   │
│   ├── models/
│   │   ├── user.py
│   │   ├── onboarding.py
│   │   ├── analysis.py
│   │   ├── task.py
│   │   ├── reflection.py
│   │   ├── progress.py
│   │   ├── tree.py
│   │   └── meditation.py
│   │
│   ├── schemas/
│   │   ├── auth.py
│   │   ├── onboarding.py
│   │   ├── analysis.py
│   │   ├── task.py
│   │   ├── reflection.py
│   │   ├── progress.py
│   │   ├── meditation.py
│   │   └── user.py
│   │
│   ├── services/
│   │   ├── auth_service.py
│   │   ├── onboarding_service.py
│   │   ├── analysis_service.py
```

```
│   │   ├── task_service.py
│   │   ├── reflection_service.py
│   │   ├── progress_service.py
│   │   └── meditation_service.py
│   │
│   ├── engines/
│   │   ├── scoring_engine.py
│   │   ├── analysis_engine.py
│   │   ├── task_generation_engine.py
│   │   ├── gamification_engine.py
│   │   ├── tree_engine.py
│   │   └── meditation_recommendation_engine.py
│   │
│   ├── integrations/
│   │   ├── gemini_client.py
│   │   └── google_auth_client.py
│   │
│   └── utils/
│       ├── constants.py
│       ├── helpers.py
│       └── prompt_templates.py
│
├── alembic/
├── tests/
├── requirements.txt
├── .env
├── alembic.ini
└── README.md
```

---

11. WHY THIS STRUCTURE IS CORRECT

This structure is correct because it clearly separates:

- route handling
- business logic

- pure decision logic
- database models
- schema validation
- external integrations

This prevents the mistakes from earlier scattered setups.

It is:

- cleaner
- easier to debug
- easier to learn
- easier to expand later

---

12. PHASE 1 MVP SCOPE (FINAL LOCK)

Phase 1 includes:

1. Google authentication
2. onboarding questions
3. onboarding analysis result
4. daily task generation
5. dashboard task flow
6. reflection flow
7. progress / streak / XP
8. tree growth logic
9. meditation recommendation system

Phase 2 later:

- community
- circles
- cohorts

- moderation dashboard
- peer support systems

---

13. PHASE 1 CORE DOMAIN ENTITIES

Phase 1 entities:

1. users
2. user_profiles
3. onboarding_sessions
4. onboarding_answers
5. analysis_results
6. tasks
7. task_logs
8. reflections
9. progress
10. tree_states
11. meditation_sessions
12. meditation_recommendations

---

14. ENTITY PURPOSE

- users
  - login identity
- user_profiles
  - baseline settings and profile information
- onboarding_sessions
  - one onboarding process instance
- onboarding_answers

    - stored answers for each question
- analysis_results
    - system interpretation of the user state
- tasks
    - daily generated actions
- task_logs
    - completion or skip history
- reflections
    - daily reflection text and mood
- progress
    - XP, streak, aggregate behavior progress
- tree_states
    - tree score and stage
- meditation_sessions
    - meditation usage records
- meditation_recommendations
    - recommended calm/focus/sleep session metadata

---

15. MAIN BACKEND MODULES

Module 1 — Auth

- Google token verification
- application JWT issuance

Module 2 — Onboarding

- question delivery
- answer submission

Module 3 — Analysis

- score processing
- user type classification
- AI summary generation

Module 4 — Tasks

- task generation
- task updates
- task retrieval

Module 5 — Reflection

- reflection storage
- reflection analysis
- obstacle detection

Module 6 — Progress

- XP updates
- streak logic
- progress summary

Module 7 — Tree

- tree score calculation
- tree stage updates

Module 8 — Meditation

- meditation category recommendation
- calm-style support mapping

---

16. FINAL AI INTEGRATION SCOPE

AI is used in only these places:

1. onboarding interpretation
2. reflection interpretation
3. task personalization
4. meditation recommendation tone

AI is not used to directly control:

- authentication
- progress truth
- XP math
- streak calculation
- tree score formula
- raw database rules

Final principle:

Rules create structure.

AI creates personalization.

---

17. RULE-BASED VS AI-BASED SPLIT

Rule-based:

- score calculation
- category aggregation
- user type thresholds
- streak updates
- XP updates
- tree score math
- task completion state
- default meditation duration

AI-based:

- summary language
- onboarding interpretation
- reflection understanding
- personalized task wording
- calm-supportive phrasing
- personalized meditation type suggestion

---

18. ONBOARDING QUESTION SYSTEM (FINAL)

The onboarding must identify:

- distraction
- discipline
- sleep / recovery
- emotional balance
- anxiety / overload
- meaning / purpose
- peace / stillness
- loneliness / connection
- digital habits

Question style:

- short
- emotionally safe
- human-like
- reflective
- non-clinical
- no neutral option

Answer scale:

- strongly agree
- agree
- disagree
- strongly disagree

---

19. LOGOTHERAPY-INSPIRED DESIGN RULE

Logotherapy should influence the system carefully.

Use themes like:

- meaning
- purpose
- responsibility
- values
- inner direction
- response to difficulty

Do not turn the onboarding into:

- intense therapy
- dark interrogation
- emotionally heavy questioning

Correct use:

- help the user discover whether they feel directionless
- help the system detect emptiness or low meaning
- support gentle movement toward meaningful action

---

20. QUESTION LANGUAGE STANDARD

Questions must sound human and calm.

Examples of good style:

- “Do you often feel distracted even when you want to focus?”
- “Do your days sometimes feel busy but not meaningful?”
- “Do you find it hard to feel calm when you are alone?”
- “Have you been feeling emotionally tired lately?”
- “Do you feel like your life is moving in a direction that matters to you?”
- “Do you often reach for your phone without thinking?”
- “Do you feel mentally rested when you wake up?”
- “Do you feel connected to people who truly understand you?”

This is the correct tone.

---

21. ONBOARDING OUTPUT (BACKEND RESULT)

After onboarding, backend must return:

- user_type
- dominant_problem
- secondary_problem
- category_scores
- short human-like summary
- first 3 daily tasks
- 1 calm / meditation recommendation
- 1 reflection prompt

This output must feel:

- clear
- warm
- useful
- non-robotic

22. USER TYPES (PHASE 1)

Recommended user types:

1. Distracted User
2. Undisciplined User
3. Anxious / Overloaded User
4. Directionless User
5. Low-Energy / Poor Recovery User
6. Balanced but Inconsistent User

Do not add too many user types in Phase 1.

Keep classification useful and stable.

23. TASK GENERATION ENGINE

The task engine must produce:

- 3 main daily tasks
- 1 optional calm task
- 1 reflection prompt

Task rules:

- small
- realistic
- time-bound
- behavior-based
- emotionally safe
- easy to act on

Example:

Distracted User

- 25 minutes of focused work
- 15 minutes away from phone
- 5 minutes calm reset

Directionless User

- write one meaningful goal for today
- complete one task that matters
- reflect on what felt meaningful today

The task engine must not generate vague motivation.

It must generate action.

---

24. REFLECTION SYSTEM

Reflection input:

- reflection text
- optional mood
- task completion state

Backend reflection processing:

- identify obstacle pattern
- identify emotional tone
- detect repeated struggles
- return short safe insight
- inform future task personalization

Reflection outputs must feel:

- calm
- human
- supportive
- practical

25. MEDITATION FEATURE (FINAL ADDITION)

Meditation is included in Phase 1.

Reason:

It is a strong competitor-standard feature and aligns with your goal of peace, stillness, and emotional reset.

This feature should be inspired by Calm-style simplicity.

Meditation categories:

1. Calm Reset
2. Focus Reset
3. Sleep Wind-Down
4. Emotional Reset
5. Meaning Reflection

Phase 1 implementation:

- static category system
- recommendation engine
- metadata-driven recommendation
- no heavy audio infrastructure yet

Important:

Meditation should feel peaceful, minimal, and restorative.

Not noisy.

Not overcomplicated.

26. SAFETY RULES

The backend must not behave like a doctor or therapist.

Safety requirements:

- no diagnosis output

- no medical claims
- no extreme labeling
- no harsh emotional interpretation
- safe fallback response for sensitive reflection content

If reflection text suggests serious distress:

- flag gently
- give safe support-oriented language
- prepare escalation/help guidance logic

Phase 1 minimum:

- keyword risk check
- safe fallback messages
- internal risk flag storage if needed

---

27. AUTHENTICATION FLOW

Auth flow:

1. User signs in with Google on frontend
2. Frontend receives Google identity token
3. Frontend sends token to backend
4. Backend verifies token
5. Backend creates or updates user
6. Backend issues app JWT
7. Frontend uses app JWT for protected routes

This is the correct auth flow for your current stack.

---

28. FINAL PHASE 1 API LIST

Auth

- POST /api/v1/auth/google

Onboarding

- GET /api/v1/onboarding/questions
- POST /api/v1/onboarding/submit
- GET /api/v1/onboarding/result/{session_id}

Tasks

- GET /api/v1/tasks/today
- POST /api/v1/tasks/{task_id}/complete
- POST /api/v1/tasks/{task_id}/skip

Reflection

- POST /api/v1/reflection
- GET /api/v1/reflection/latest

Progress

- GET /api/v1/progress
- GET /api/v1/tree

Meditation

- GET /api/v1/meditation/recommendation
- GET /api/v1/meditation/categories

---

29. STANDARD API RESPONSE FORMAT

Success response:

```
{
  "success": true,
  "data": {},
  "message": "string"
}
```

Error response:

```
{
  "success": false,
  "message": "string",
  "error": "string"
}
```

---

30. FINAL BUILD ORDER

STEP 1

- FastAPI setup
- environment config
- database connection
- SQLAlchemy base
- Alembic setup

STEP 2

- user model
- Google auth verification
- JWT issuance

STEP 3

- onboarding question schema
- onboarding session model
- onboarding answers model
- onboarding routes

STEP 4

- scoring engine
- analysis engine

- Gemini client
- onboarding result endpoint

STEP 5

- task model
- task generation engine
- tasks/today route

STEP 6

- reflection model
- reflection route
- reflection analysis

STEP 7

- progress model
- gamification engine
- tree engine
- progress/tree routes

STEP 8

- meditation recommendation engine
- meditation API

---

31. USEFUL DECISIONS RETAINED AND CORRECTED

Retained from earlier backend direction:

- backend as system brain
- layered logic
- task engine
- progress engine
- tree engine

- backend as source of truth
- standardized responses

Corrected for final version:

- language is Python, not Node backend
- validation is Pydantic, not Zod
- no early Redis/cache/jobs complexity
- AI endpoints are simplified and focused
- meditation added as a core feature
- human-like language is mandatory across app outputs

---

32. FINAL JUDGMENT

This document is now aligned with:

- your selected language
- your selected framework
- your learning level
- your MVP scope
- your long-term product vision
- your need for a clean rebuild
- your requirement for human-like user language
- your requirement for meaning, peace, calm, and purpose support
- your requirement to include meditation as a key feature

This is the final backend foundation direction.

---

33. NEXT STEP

The correct next document is:

FINAL POSTGRESQL DATABASE SCHEMA DOCUMENT

Use this command next:

"Reet, generate the final PostgreSQL database schema document in structured snippet format for Google AI Studio."

END OF FINAL DOCUMENT